

```
pip install matplotlib
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
```

```
import tensorflow as tf # TensorFlow library for deep learning
from tensorflow.keras.datasets import mnist # MNIST handwritten digits dataset
from tensorflow.keras.models import Sequential # Sequential model (layer by layer)
from tensorflow.keras.layers import Dense # Fully connected (Dense) layer
from tensorflow.keras.utils import to_categorical # Convert labels to one-hot format
import matplotlib.pyplot as plt # For plotting images/graphs
import numpy as np # Numerical operations
```

```
#
# 1. DATA ACQUISITION
#
print("Loading data...")
# Load the MNIST dataset (60,000 training images, 10,000 test images)
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Save original test images for display
X_test_images = X_test
```

```
Loading data...
```

```
X_train = X_train.reshape(-1, 784) / 255.0
X_test = X_test.reshape(-1, 784) / 255.0
```

```
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

```
model = Sequential([
    Dense (128, activation='relu', input_shape=(784,)),
    Dense (64, activation='relu'),
    Dense (10, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
print("\nStarting training...")
history = model.fit(
    X_train, y_train_cat,
    epochs=50,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)
```

```
Epoch 27/50
```

```

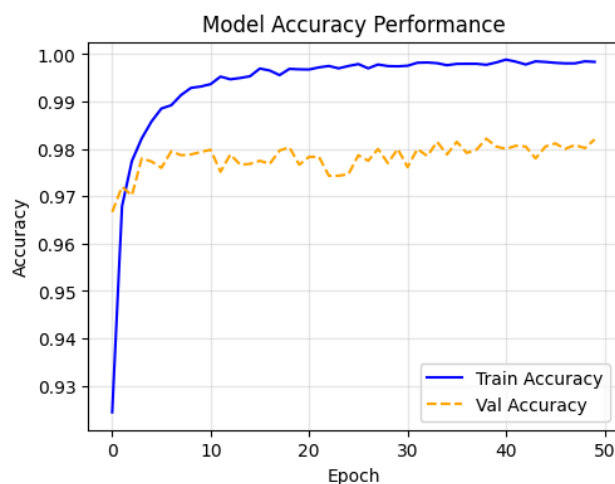
1688/1688 [=====] - 6s 3ms/step - loss: 0.0103 - accuracy: 0.9970 - val_loss: 0.1470 - val_accu
Epoch 28/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0065 - accuracy: 0.9978 - val_loss: 0.1415 - val_accu
Epoch 29/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0079 - accuracy: 0.9975 - val_loss: 0.1655 - val_accu
Epoch 30/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0087 - accuracy: 0.9974 - val_loss: 0.1445 - val_accu
Epoch 31/50
1688/1688 [=====] - 6s 4ms/step - loss: 0.0080 - accuracy: 0.9976 - val_loss: 0.1495 - val_accu
Epoch 32/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0065 - accuracy: 0.9982 - val_loss: 0.1530 - val_accu
Epoch 33/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0060 - accuracy: 0.9983 - val_loss: 0.1502 - val_accu
Epoch 34/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0067 - accuracy: 0.9981 - val_loss: 0.1620 - val_accu
Epoch 35/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0071 - accuracy: 0.9977 - val_loss: 0.1674 - val_accu
Epoch 36/50
1688/1688 [=====] - 6s 4ms/step - loss: 0.0068 - accuracy: 0.9980 - val_loss: 0.1461 - val_accu
Epoch 37/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0067 - accuracy: 0.9980 - val_loss: 0.1683 - val_accu
Epoch 38/50
1688/1688 [=====] - 6s 4ms/step - loss: 0.0068 - accuracy: 0.9980 - val_loss: 0.1613 - val_accu
Epoch 39/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0079 - accuracy: 0.9978 - val_loss: 0.1413 - val_accu
Epoch 40/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0061 - accuracy: 0.9983 - val_loss: 0.1541 - val_accu
Epoch 41/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0047 - accuracy: 0.9989 - val_loss: 0.1536 - val_accu
Epoch 42/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0058 - accuracy: 0.9984 - val_loss: 0.1417 - val_accu
Epoch 43/50
1688/1688 [=====] - 6s 3ms/step - loss: 0.0071 - accuracy: 0.9978 - val_loss: 0.1617 - val_accu
Epoch 44/50

```

```

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy', color='blue')
plt.plot(history.history['val_accuracy'], label='Val Accuracy', color='orange', linestyle='--')
plt.title('Model Accuracy Performance')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss', color='red')
plt.plot(history.history['val_loss'], label='Val Loss', color='darkred', linestyle='--')
plt.title('Model Error (Loss) Trend')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```



```

print("\nGenerating predictions on unseen test data...")
predictions = model.predict(X_test)
plt.figure(figsize=(15, 3))
for i in range(10):
plt.subplot(1, 10, i+1)

```

```
plt.imshow(X_test_images [i], cmap='gray')
pred_label = np.argmax(predictions [i])
true_label = y_test[i]
color = 'green' if pred_label == true_label else 'red'
plt.title(f"Pred: {pred_label}\nTrue: {true_label}", color=color, fontsize=10)
plt.axis('off')

plt.tight_layout()
plt.show()
```

```
File "/tmp/ipython-input-3301523066.py", line 5
  plt.subplot(1, 10, i+1)
    ^
IndentationError: expected an indented block after 'for' statement on line 4
```

Next steps: [Explain error](#)

Start coding or [generate](#) with AI.